2. Avoid global variables (variables that begin with a dollar sign). If you need some data globally available, assign it to a constant and freeze its value so that you'll at least get a warning if you try to modify it (multiple threads reading the same data is generally safe).

3. Avoid methods that modify class variables, like the `Fulfiller` above. Any code that stores data in a class variable and provides public methods to change that data is potentially thread unsafe.

4. If you must have shared global data that threads must modify, use the Concurrent Ruby gem,[22] which allows you to wrap access to shared data in thread-safe methods.

5. If you have multiple jobs manipulating the same underlying data in your database, consider serializing those jobs so they run one at a time (we'll see how to do that later in this chapter), or using the locking features of your database to create a more predictable environment.

Preventing concurrency issues that extend beyond your app's codebase can be extremely difficult. Just know that having a Rails app with Sidekiq jobs means that you have a distributed system. The monitoring and alerting discussed in the previous section is the key to managing it. Any concurrency issue *should* show up as a problem with your business function measurements, and the other measures can help you triangulate what may be the cause.

Adjust Concurrency Carefully

Once you've avoided common concurrency issues in your code, you may want to adjust the number of threads or the number of processes, or both. You can do this three ways, depending on your needs.

The first is what was alluded to in *Set Sidekiq's Configuration Options*, which is to modify the environment variable `SIDEKIQ_CONCURRENCY` in your production environment. That controls the number of threads per process. Make only small changes and observe their effects. More threads consume more memory, but each thread will also consume a database connection, and those are typically limited.